

specific scene. For example,

P1: `Filter(Red, Relate(Left, Filter(Sphere)))` and  
 P2: `Filter(Red, Relate(Left, Filter(Cube)))`

may refer to the same red object in a specific scene. Motivated by the REINFORCE process, to suppress such spurious programs, we use the loss function:

$$\ell = \sum_{p \in \mathbb{Q}} \text{stop\_gradient}(\Pr[p]) \cdot (-\log \Pr[p]).$$

The corresponding gradient  $\nabla_{\Theta_s}$  is,

$$\nabla_{\Theta_s} = \sum_{p \in \mathbb{Q}} \Pr[p] \cdot \nabla_{\Theta_s} (r \cdot \log \Pr[p]) = \nabla_{\Theta_s} \left( \sum_{p \in \mathbb{Q}} r \cdot \Pr[p] \right).$$

The key observation is that, given a sufficiently large set of scenes, a program can be identified as spurious if there exists at least one scene where the program leads to a wrong answer. As the training goes, spurious programs will get less update due to the sampling importance term  $\Pr[p]$  which weights the likelihood maximization term.

## E CURRICULUM LEARNING SETUP

During the whole training process, we gradually add more visual concepts and more complex question examples into the model. Summarized in Figure 4(A), in general, the whole training process is split into 3 stages. First, we only use questions from lesson 1 to let the model learn object-level visual concepts. Second, we train the model to parse simple questions and to learn relational concepts. In this step, we freeze the neural operators and concept embeddings of object-level concepts. Third, the model gets trained on the full question set (lesson 3), learning to understand questions of different complexities and various format. For the first several iterations in this step, we freeze the parameters in the perception modules. In addition, during the training of all stages, we gradually increase the number of objects in the scene: from 3 to 10.

We select questions for each lesson in the curriculum learning by their depth of the latent program layout. For example, the program `Query(Shape, Filter(Red, Scene))` has the depth of 3, while the program `Query(Shape, Filter(Cube, Relate(Left, Filter(Red, Scene))))` has the depth of 5. Since we have fused consecutive `Filter` operations into a single one, the maximum depth of all programs is 9 on the CLEVR dataset. We now present the detailed split of our curriculum learning lessons:

For lesson 1, we use only programs of depth 3. It contains three types of questions: querying an attribute of the object, querying the existence of a certain type of objects, count a certain type of objects, and querying if two objects have the same attribute (e.g., of the same color). These questions are almost about fundamental object-based visual concepts. For each image, we generate 5 questions of lesson 1.

For lesson 2, we use programs of depth less than 5, containing a number of questions regarding relations, such as querying the attribute of an object that is left of another object. We found that in the original CLEVR dataset, all `Relate` operations are followed by a `Filter` operation. This setup degenerates the performance of the learning of relational concepts such as `Left`. Thus, we add a new question template into the original template set: `Count(Relate( $\cdot$ , Filter( $\cdot$ , Scene)))` (e.g., “What’s the number of objects that are left of the cube?”). For each image, we generate 5 questions of lesson 2.

For lesson 3, we use the full CLEVR question set.

Curriculum learning is crucial for the learning of our neuro-symbolic concept learner. We found that by removing the curriculum setup w.r.t. the number of object in the scenes, the visual perception module will get stuck at an accuracy that is similar to a random-guess model, even if we only use stage-1 questions. If we remove the curriculum setup w.r.t. the complexity of the programs, the joint training of the visual perception module and the semantic parser can not converge.

## F ABLATION STUDY

We conduct ablation studies on the accuracy of semantic parsing, the impacts of the ImageNet pre-training of visual perception modules, the data efficiency of our model, and the usage of object-based representations.

### F.1 SEMANTIC PARSING ACCURACY.

We evaluate how well our model recovers the underlying programs of questions. Due to the intrinsic equivalence of different programs, we evaluate the accuracy of programs by executing them on the ground-truth annotations of objects. Invalid or ambiguous programs are also considered as incorrect. Our semantic parser archives  $> 99.9\%$  QA accuracy on the validation split.

### F.2 IMPACTS OF THE IMAGENET PRE-TRAINING.

The only extra supervision of the visual perception module comes from the pre-training of the perception modules on ImageNet (Deng et al., 2009). To quantify the influence of this pre-training, we conduct ablation experiments where we randomly initialize the perception module following He et al. (2015). The classification accuracies of the learned concepts almost remain the same except for Shape. The classification accuracy of Shape drops from 98.7 to 97.5 on the validation set while the overall QA accuracy on the CLEVR dataset drops to 98.2 from 98.9. We speculate that large-scale image recognition dataset can provide prior knowledge of shape.

### F.3 DATA EFFICIENCY AND OBJECT-BASED REPRESENTATIONS

In this section, we study whether and how the number of training samples and feature representations affect the overall performance of various models on the CLEVR dataset. Specifically, we compare the proposed NS-CL against two strong baselines: TbD (Mascharka et al., 2018) and MAC (Hudson & Manning, 2018).

**Baselines.** For comparison, we implement two variants of the baseline models: TbD-Object and MAC-Object. Inspired by Anderson et al. (2018), instead of using a 2D convolutional feature map, TbD-Object and MAC-Object take a stack of object features as inputs, whose shape is  $k \times d_{obj}$ .  $k$  is the number of objects in the scene, and  $d_{obj}$  is the feature dimension for a single object. In our experiments, we fix  $k = 12$  as a constant value. If there are fewer than 12 objects in the scene, we add “null” objects whose features are all-zero vectors.

We extract object features in the same way as NS-CL. Features are extracted from a pre-trained ResNet-34 network before the last residual block for a feature map with high resolution. For each object, its feature is composed of two parts: region-based (by RoI Align) and image-based features. We concatenate them to represent each object. As discussed, the inclusion of the representation of the full scene is essential for the inference of relative attributes such as size or spatial position on the CLEVR domain.

TbD and MAC networks are originally designed to use image-level attention for reasoning. Thus, we implement two more baselines: TbD-Mask and MAC-Mask. Specifically, we replace the original attention module on images with a mask-guided attention. Denotes the union of all object masks as  $M$ . Before the model applies the attention on the input image, we multiply the original attention map computed by the model with this mask  $M$ . The multiplication silences the attention on pixels that are not part of any objects.

**Results.** Table 3 summarizes the results. We found that TbD-Object and MAC-Object approach show inferior results compared with the original model. We attribute this to the design of the network architectures. Take the `Relate` operation (e.g., finds all objects left of a specific object  $x$ ) as an example. TbD uses a stack of dilated convolutional layers to propagate the attention from object  $x$  to others. In TbD-Object, we replace the stack of 2D convolutions by several 1D convolution layers, operating over the  $k \times d_{obj}$  object features. This ignores the equivalence of objects (the order of objects should not affect the results). In contrast, MAC networks always use the attention mechanism

to extract information from the image representation. This operation is invariant to the order of objects, but is not suitable for handling quantities (e.g., counting objects).

As for TbD-Mask and MAC-Mask, although the mask-guided attention does not improve the overall performance, we have observed noticeably faster convergence during model training. TbD-Mask and MAC-Mask leverage the prior knowledge of object masks to facilitate the attention. Such prior has also been verified to be effective in the original TbD model: TbD employs an attention regularization during training, which encourages the model to attend to smaller regions.

In general, NS-CL is more data-efficient than MAC networks and TbD. Recall that NS-CL answers questions by executing symbolic programs on the learned visual concepts. Only visual concepts (such as *Red* and *Left*) and the interpretation of questions (how to translate questions into executable programs) need to be learned from data. In contrast, both TbD and MAC networks need to additionally learn to execute (implicit or explicit) programs such as counting.

For the experiments on the full CLEVR training set, we split 3,500 images (5% of the training data) as the hold-out validation set to tune the hyperparameters and select the best model. We then apply this model to the CLEVR validation split and report the testing performance. Our model reaches an accuracy of 99.2% using the CLEVR training set.

## G EXTENDING TO OTHER SCENE AND LANGUAGE DOMAINS

### G.1 MINECRAFT DATASET

We also extend the experiments to a new reasoning testbed: Minecraft worlds (Yi et al., 2018). The Minecraft reasoning dataset differs from CLEVR in both visual appearance and question types. Figure 9 gives an example instance from the dataset.



Q: What direction is the closest creature facing?  
A: Left.  
P: Query(Direction, FilterMost(Closest,  
Filter(Creature)  
))

Figure 9: An example image and a related question-answering pair from the Minecraft dataset.

**Setup.** Following Yi et al. (2018), we generate 10,000 Minecraft scenes using the officially open-sourced tools by Wu et al. (2017). Each image contains 3 to 6 objects. The objects are chosen from 12 categories, with 4 different facing directions (front, back, left and right). They stand on a 2D plane.

Besides different 3D visual appearance and image contexts, the Minecraft reasoning dataset introduces two new types of reasoning operations. We add them to our domain-specific language:

1. `FilterMost(ObjectSet, Concept) → ObjectSet`: Given a set of objects, finds the “most” one. For example, `FilterMost(Closest, set)` locates the object in the input set that is closest to the camera (e.g., what is the direction of the closest animal?)
2. `BelongTo(Object, ObjectSet) → Bool`: Query if the input object belongs to a set.

**Results.** Table 10 summarizes the results and Figure 12 shows sample execution traces. We compare our method against the NS-VQA baseline (Yi et al., 2018), which uses strong supervision for both scene representation (e.g., object categories and positions) and program traces. In contrast, our method learns both by looking at images and reading question-answering pairs. NS-CL outperforms NS-VQA by 5% in overall accuracy. We attribute the inferior results of NS-VQA to its derendering module. Because objects in the Minecraft world usually occlude with each other, the detected object bounding boxes are inevitably noisy. During the training of the derendering module, each detected bounding box is matched with one of the ground-truth bounding boxes and uses its class and pose as supervision. Poorly localized bounding boxes lead to noisy labels and hurt the accuracy of the derendering module. This further influences the overall performance of NS-VQA.